CS 22403 Design and Analysis of Algorithm

1) Solve below function:

If $T_1(n) = O(f(n))$ and $T_2(n) = O(g(n))$, then show that

a) $T_1(n) + T_2(n) = \max(O(f(n)), O(g(n))$

b) $T_1(n) * T_2(n) = O(f(n) * g(n))$

Given

$T_2(n) = O(g(n))$

a) proof

By definition of Big-O

$T_1(n) \leq C_1 f(n)$

$T_2(n) \leq C_2 g(n)$

Add both

$T_1(n) + T_2(n) \leq C_1 f(n) + C_2 g(n)$

Let $h(n) = \max(f(n), g(n))$

so,

$C_1 f(n) + C_2 g(n) \leq (C_1 + C_2) h(n)$

Hence

$T_1(n) + T_2(n) = O(\max(f(n), g(n)))$

proved.

b) $T_1(n) * T_2(n) = O(f(n) * g(n))$

from defn:

$T_1(n) \leq C_1 f(n)$, $T_2(n) \leq C_2 g(n)$

Multiply:

$T_1(n) * T_2(n) \leq C_1 C_2 f(n) g(n)$

Therefore

$T_1(n) * T_2(n) = O(f(n) \times g(n))$

proved.

(ii) Derive the Mathematical analysis to compute the efficie

a) finding factorial of 'n' integers

Algorithm: # finding factorial of n integers

```
fact = 1
for i = 1 to n:
    fact = fact * i
```

Loop runs n times

each step is constant time

time complexity = $O(n)$

space complexity = $O(1)$

b) To counting binary digits

To count the binary digits

```
count = 0
while n > 0:
    n = n/2
    count ++
```

No of iterations = no of bits = $\log_2 n$

Time complexity = $O(\log n)$

space complexity = $O(1)$

c) # finding unique element in an array

compare each element with others

two nested loops $\rightarrow n \times n$

Time complexity = $O(n^2)$

space complexity = $O(1)$

(using hashing $\rightarrow O(n)$, but basic analysis expects $O(n^2)$)

...ent record Management system

...) Explain how insertion sort operates in the given scenario

* students join one by one
* Each new roll numbers is inserted into its correct position
* Already admitted students are kept sorted
* similar to arranging cards in hand.

(ii) Time complexity of insertion sort

case	situation	time
Best	Already sorted	$O(N)$
Average	Random order	$O(N^2)$
worst	Reverse order	$O(N^2)$

(iii) Efficiency as no of students increases

* small number of students → works fast
* Large number → shifting cost increases
* Time grows → quadratically (N^2)

(iv) Is insertion sort suitable?

yes, suitable when:

* students are admitted gradually.
* data size is small / moderate
* Mostly sorted already

Not suitable when:

* very large no of students

3) Apply Masters Theorem:

a) $T(n) = 3T(n/2) + n^2$

G.f

$T(n) = aT(n/b) + f(n)$

$T(n) = 3T(n/2) + n^2$

$a = 3$

$n^{\log_b a} = n^{\log_2 3} \approx n^{1.58}$

since $n^2 > n^{1.58}$

$T(n) = O(n^2)$

b) $T(n) = 4T(n/2) + n^2$

$n^{\log_2 4} = n^2$

$T(n) = O(n^2 \log n)$

c) $T(n) = T(n/2) + 2^n$

$n^{\log_2 1} = n^0 = 1$

2^n dominates

case 3:

$T(n) = O(2^n)$

d) $T(n) = 16T(n/4) + n$

$n^{\log_4 16} = n^2$

case 1:

$T(n) = O(n^2)$

...ting using Moving intervals:

Ans:

problem understanding:

* There a c cakes arranged in a row (no 1 to c)
* There are n children
* Each child choose a continuous interval of cakes [Si, Ei]
* If two children choose overlapping intervals, the will fight.
    * you are given an integer K
    * K=0 -> No child can change decision
    * K=1 => you can persuade at most one child to change interva
your task is to print:
    * "Good" -> if no fight happens
    * "Bad" -> if a fight is unavoidable

case 1: when K=0

idea
    * no changes allowed
    * Just check whether any two intervals overlap

Algorithm:
1. sort all intervals based on starting value Si
2. compare consecutive intervals
3. if Si+1 ≤ Ei, then overlap exists
4. if overlap found -> print "Bad"
5. If no overlap -> print "Good"

Time complexity

* sorting : $O(n \log n)$
* checking overlap : $O(n)$
* overall : $O(n \log n)$

case 2: when $K=1$

* you can modify only one child's interval
* Length must remain same

$(E_i - S_i + 1) = (y - x + 1)$

steps

1- first, check overlaps in original intervals.
2- If no overlap → print "Good"
3. If overlap exists.
Try changing one child interval
shift the interval left or right without changin its length.
4- After change.
5- If overlap removed → "Good"
6- If overlap still exists in all ~~cases~~ cases → "Bad"

Explanation of Given samples:

eg: 1

$C=5 \quad N=2 \quad K=0$

Intervals:

2 2

5 5

* child 1 eats cake 2
* child 2 eats cakes 3, 4, 5
* No overlap

Answer: Good

eg: 2

C = 5 N = 2 K = 1

Intervals

2 3

2 5

* overlap at cakes 2 and 3
* Try changing one child's interval
* Any change still causes overlap

Answer: Bad

final conclusion:

* if K=0, simply check for overlapping intervals
* if K=1, allow at most one interval modification while presenting length.
* If conflicts can be resolved → Good
* otherwise → bad.